

AI Declaration

Provider: OpenAI

Model: ChatGPT

Version: 5.5 (Instant and thinking)

Prompts

1.

I need help designing a substantial Assignment 6 mini-project for a Foundations of Digital Infrastructure course.

Important context:

- * The project must be significantly more substantial than a simple benchmark or measurement experiment.
- * A previous idea involving Monte Carlo option pricing and measuring runtime/energy was considered too small.
- * The assignment examples include things like a Performance Bottleneck Detector, Network Sustainability Tracker, and ML Sustainability Estimator.
- * I am interested in quantitative finance, optimisation, machine learning, sustainability, performance measurement, and resource allocation.
- * I need something that can realistically be built in about one day, but should still feel like a genuine project rather than a coursework exercise.

Current direction:

I want to build a Level-2++ ML Sustainability Estimator / Model Recommendation System.

The goal is NOT:

- * train a few sklearn models,
- * compare accuracy,
- * pick the winner.

That is too trivial.

Instead, I want a genuine optimisation and decision-making system.

The system should:

1. Train multiple ML models on one or more datasets.
2. Measure:

- * Accuracy
 - * Training time
 - * Inference time
 - * Energy consumption
 - * Estimated carbon footprint
3. Build performance profiles for the models.
 4. Accept user constraints such as:
 - * Minimum accuracy
 - * Maximum runtime
 - * Maximum energy budget
 - * Maximum carbon budget
 - * Preferred operating mode
 5. Automatically recommend the optimal model.

I want the recommendation engine to be the centrepiece of the project.

I would like the project to include concepts such as:

- * Multi-objective optimisation
- * Pareto frontiers
- * Tradeoffs between performance and sustainability
- * Constraint satisfaction
- * Resource-aware decision making

I do NOT want a toy weighted-score system that always picks the same model.

I want the recommendation to genuinely change depending on the user's constraints.

IMPORTANT:

I do not want you to write code for me.

I do not want implementation details, code snippets, pseudocode, class structures, function definitions, or step-by-step programming instructions.

I want help with:

- * Project design
- * System architecture ideas
- * What makes a project intellectually interesting
- * What optimisation formulations would be meaningful
- * What datasets would be suitable
- * What model families would be interesting to compare
- * What constraints users should be able to specify
- * What outputs would make the system useful
- * What visualisations would strengthen the project

- * What analyses would strengthen the report
- * What discussion points would impress a marker
- * What extensions would make the project feel more substantial

Please focus on helping me design the project at a conceptual level.

Please challenge weak ideas and suggest stronger alternatives.

I am looking for guidance on what should be included, what should be measured, what the recommendation engine should consider, what the report structure should be, and what features would make this feel like a genuinely strong mini-project rather than a simple benchmark. Understand all this then i

2.

{in reference to carbon intensity scenario, * batch size or deployment setting, * interpretability requirement,}
what is practically meant by these

3.

Ok, what is the best way to build the structure of this project, with the different files and folder, remember i dont want you to code anything for me

4.

ok now i just want very general steps that i should take when programming this project, like what does progress look like for this project, where to start, steps to take, milestones to achieve desired outcome. Remember no code, keep things general i just need a direction to go in. Im thinking like load data -> load models of choice - > train - ...

5.

would this be an ok one to start with: Context

It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that they did not purchase.
Content

The dataset contains transactions made by credit cards in September 2013 by European cardholders.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

It contains only numerical input variables which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, we cannot provide the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with

PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise.

Given the class imbalance ratio, we recommend measuring the accuracy using the Area Under the Precision-Recall Curve (AUPRC). Confusion matrix accuracy is not meaningful for unbalanced classification.

6.

since the dataset has 10 columns is it still possible to train it using these models: Linear model

Tree based model

Ensemble model

Margin based model

SVM

Random Forest

Small neural network

in python, dont want you to code anything for me

7.

so these ML models im trying to train, what is the general process of, actually getting them an training them

8.

ive heard about pytorch being big, what is the difference in using that vs sklearn

9.

Can you give me a lesson of how to use sklearn, dont go into complete detail but teach me enough to do this project easily, here is a sample of my data and description: Context It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that they did not purchase. Content The dataset contains transactions made by credit cards in September 2013 by European cardholders. This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions. It contains only numerical input variables which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, we cannot provide the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. Given the class imbalance ratio, we recommend measuring the accuracy using the Area Under the Precision-Recall Curve (AUPRC). Confusion matrix accuracy is not meaningful for unbalanced classification.

10.

how does the sklearn react with panda dataframes, like do i have to just pass in the values when i do the model.fit(..) or can i pass something in like this `Y = df["Class"]` with the header and all

11.

What are they doing here, what is this pipeline thing, is it something i am meant to implement, can it be used for all of the models to keep a level playing field: Pipelines: chaining pre-processors and estimators

Transformers and estimators (predictors) can be combined together into a single unifying object: a Pipeline. The pipeline offers the same API as a regular estimator: it can be fitted and used for prediction with fit and predict. As we will see later, using a pipeline will also prevent you from data leakage, i.e. disclosing some testing data in your training data.

In the following example, we load the Iris dataset, split it into train and test sets, and compute the accuracy score of a pipeline on the test data:

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# create a pipeline object
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression()
)

# load the iris dataset and split it into train and test sets
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

# fit the whole pipeline
pipe.fit(X_train, y_train)
Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('logisticregression', LogisticRegression())])
```

12.

any drastic problems with this: `def split(filepath: str) -> Matrix, Vector:`
`df = load_data(filepath)`

```
# Splits the dataframe into the training vector and the target vector
X = df.drop("Class", axis =1) # training vector
y = df["Class"] #target vector
```

```

#Will split up the data into a train section and a train section
#Tests the model on 20% of the data, random_state = 42 to get the same results each
>
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_st>

return X_train, X_test, y_train, y_test

def LogRegression(filepath: str = "../data/creditcard.csv"):

    # Will train a logistic regression model on the data

    X_train, X_test, y_train, y_test = split(filename)

    start_train = time.time()

    # creates the pipeline object, this combines the preprocessing and the model
    #The standard scaler will remove the mean and scale with a unit variance
    pipe= make_pipeline(
        StandardScaler(),
        LogisticRegression(),
    )

    logistic = pipe.fit(X_train, y_train)

    end_train = time.time()
    train_time = end_train - start_train

    return logistic, train_time, X_test, y_test

```

13.

what does max_iter mean and what would be a good value

14.

iss this ok, can i implement the next ones now: def split(filepath: str) -> tuple:

```

df = load_data(filepath)

# Splits the dataframe into the training vector and the target vector
X = df.drop("Class", axis =1) # training vector
y = df["Class"] #target vector

#Will split up the data into a train section and a train section
#Tests the model on 20% of the data, random_state = 42 to get the same results each
>
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_st>

return (X_train, X_test, y_train, y_test)

```

```
def LogRegression(filepath: str = "../data/creditcard.csv"):

    # Will train a logistic regression model on the data

    X_train, X_test, y_train, y_test = split(filepath)

    start_train = time.time()

    # creates the pipeline object, this combines the preprocessing and the model
    #The standard scaler will remove the mean and scale with a unit variance
    #increased iteration count to allow convergence
    #balanced uses the values of y to adjust the weights inversely proportional to frequ>
    pipe= make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter = 1000,class_weight="balanced" ),
    )

    logistic = pipe.fit(X_train, y_train)

    end_train = time.time()
    train_time = end_train - start_train

    return (logistic, train_time, X_test, y_test)
```

15.

What is the difference in the decision tree regression and the classification and which would be best for what i am building

16.

any issues or info im not taking into account: def split(filepath: str) -> tuple:

```
df = load_data(filepath)

# Splits the dataframe into the training vector and the target vector
X = df.drop("Class", axis =1) # training vector
y = df["Class"] #target vector

#Will split up the data into a train section and a test section
#Tests the model on 20% of the data, random_state = 42 to get the same results each
>
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_st>

return (X_train, X_test, y_train, y_test)

def DecisionTree(filepath: str = "../data/creditcard.csv"):
```

```
# Will train a logistic regression model on the data
```

```
X_train, X_test, y_train, y_test = split(filepath)
```

```
start_train = time.time()
```

```
tree = tree.DecisionTreeClassifier(class_weight = "balanced").fit(X_train, y_train)
```

```
end_train = time.time()
```

```
train_time = end_train - start_train
```

```
return (tree, train_time, X_test, y_test)
```

17.

{referencing: `max_depth`, `min_samples_leaf`, or `min_samples_split`}

explain please and do i need to do the random state thing if i have already split up the data in the previous function?

18.

ok: def SupportVectorMachine(filepath: str = "../data/creditcard.csv"):

```
# Will train a logistic regression model on the data
```

```
X_train, X_test, y_train, y_test = split(filepath)
```

```
start_train = time.time()
```

```
pipe = make_pipeline(
```

```
    StandardScaler(),
```

```
    svm.LinearSVC(class_weight = "balanced", random_state = 42, max_iter = 5000)
```

```
)
```

```
#trains the model on the data, cap the tree depth to avoid overfitting
```

```
model = pipe.fit(X_train, y_train)
```

```
end_train = time.time()
```

```
train_time = end_train - start_train
```

```
return (model, train_time, X_test, y_test)
```

19.

can you explain what is going on here and how to interpret this AUPRC curve

20.

why do i need to get all these other metrics for accuracy, surely just the AUPRC is fine

21.

```
def get_accuracy(model, X_test, y_test):
    start_inference = time.time()

    y_scores = model.predict_proba(X_test)

    end_inference = time.time()
    inference_time = end_inference - start_inference

    accuracy = average_precision_score(y_test, y_scores)

    return accuracy, inference_time
```

22.

so for the energy stuff i wanted to use the sudo turbostat thing but inside the actual code, do you know k=how i could do that

23.

i feel like we should add into recommend model, how much inference the user is planning on doing because with a large enough number some rankings will flop, like the total time of the lr is usually less than the svm but for enough inference, since the inference time on the svm is less than that of the lr